STUDENT PERFORMANCE ANALYTICS SYSTEM

Python with AI – Mini Project Documentation

Student: Syeda Sarah Parvin

Course: Python with AI

Technology: Python, NumPy, Pandas

Project: Student Performance Analytics System

GitHub Repository: https://github.com/Uhan-ara/student-performance-analytics

1. Project Title

Student Performance Analytics System

The Student Performance Analytics System is a Python-based data analysis project developed to process and analyze student academic performance.

The system uses **Python, NumPy and Pandas** to load student records, calculate academic performance metrics, assign grades, determine pass/fail status and generate useful class and subject-wise insights.

The project demonstrates the practical use of Python fundamentals including **variables, data types, loops, functions, conditions, NumPy, Pandas and data analysis**.

2. Objective

The main objective of this project is to develop a simple and interactive system that can analyze student academic performance from a structured dataset.

The system is designed to:

- Load student data from a CSV file.
- Organize the data using a Pandas DataFrame.
- Calculate total marks for each student.
- Calculate average marks for each student.
- Assign grades according to a defined grading system.
- Determine whether each student has passed or failed.
- Calculate overall class performance.
- Perform subject-wise performance analysis.
- Identify the top-performing students.
- Analyze student attendance.
- Search for an individual student's performance using Student ID.

The project provides a practical way to understand student performance without manually calculating each metric.

3. Technologies Used

Python

Python is used as the main programming language for implementing the application logic, functions, loops, conditions, calculations and interactive menu.

Pandas

Pandas is used for:

- Reading the CSV dataset.
- Creating and manipulating the DataFrame.
- Filtering student records.
- Sorting students.
- Performing statistical analysis.
- Generating class and subject-wise results.

NumPy

NumPy is used for numerical calculations such as:

- Calculating the sum of subject marks.
- Calculating average marks.
- Finding minimum subject marks.
- Performing numerical operations efficiently.

CSV

A CSV file is used to store the student dataset in a structured tabular format.

4. Dataset Description

The project uses a realistic dataset containing **25 student records**.

The students belong to three departments:

- Computer Science and Engineering (CSE)
- Information Technology (IT)
- Electronics and Communication Engineering (ECE)

The dataset contains the following columns:

Column	Description
Student_ID	Unique identification number of the student
Name	Student's name
Department	Student's department
Attendance	Attendance percentage
Column	**Description**
Python	Marks obtained in Python
Mathematics	Marks obtained in Mathematics
Data_Analysis	Marks obtained in Data Analysis

AI	Marks obtained in Artificial Intelligence
Database	Marks obtained in Database

The dataset contains **five subject marks** for each student along with attendance information.

The dataset satisfies the project requirement of using at least 15–20 realistic student records.

5. Implementation

Step 1 – Loading the Dataset

The student records are stored in `students.csv`.

Pandas is used to read the CSV file and create a DataFrame.

The DataFrame allows the program to easily access, filter, sort and analyze student information.

Step 2 – Calculating Student Metrics

For every student, the system calculates:

- Total marks
- Average marks
- Grade
- Pass/Fail result

The calculations are implemented through reusable functions in `functions.py`.

Step 3 – Total Marks

The total marks are calculated by adding the marks obtained in all five subjects.

NumPy's `sum()` function is used for the numerical calculation.

Step 4 – Average Marks

The average is calculated from the five subject marks.

NumPy's `mean()` function is used to calculate the average.

The result is rounded to two decimal places.

Step 5 – Grade Assignment

The system assigns grades based on the student's average marks.

The grading system used is:

Average Marks	Grade
90 and above	A+
80–89.99	A
Average Marks	**Grade**
70–79.99	B
60–69.99	C
50–59.99	D
Below 50	F

Step 6 – Pass/Fail Condition

A student is considered **Pass** when: •

Every subject mark is at least 40, and

• Attendance is at least 75%.

If either condition is not satisfied, the student is marked **Fail**.

This condition is implemented using Python conditional statements and NumPy's minimum-value calculation.

Step 7 – Class Performance Analysis

The system calculates:

- Total number of students
- Number of students who passed
- Number of students who failed
- Pass percentage
- Overall class average
- Highest average
- Lowest average

Step 8 – Subject-wise Analysis

A loop is used to process each subject.

For every subject, the system calculates:

- Average marks
- Highest marks
- Lowest marks

This allows comparison of performance across different subjects.

Step 9 – Top Performers

The system sorts students according to their average marks in descending order.

The **Top 5 Performers** are then displayed along with their:

- Student ID
- Name
- Department
- Average
- Grade
- Result

Step 10 – Attendance Analysis

The system analyzes attendance by calculating:

- Average attendance
- Highest attendance
- Lowest attendance
- Students below the 75% attendance requirement

Step 11 – Student Search

The user can enter a Student ID.

The system searches the DataFrame and displays the corresponding student's complete performance information.

Step 12 – Interactive Menu

The application provides a menu-driven interface with the following options:

1. View Student Data
2. Class Performance Summary
3. Subject-wise Analysis
4. Top 5 Performers
5. Attendance Analysis
6. Search Student
7. Exit

This makes the system easy to operate without requiring the user to inspect the Python code.

6. Key Features

The major features implemented in the project are:

- CSV-based student dataset
- 25 student records
- Pandas DataFrame processing
- NumPy numerical calculations
- Reusable Python functions
- Loops for repeated analysis
- Conditional statements for grading and results
- Total marks calculation
- Average marks calculation
- Automatic grade assignment
- Pass/fail determination
- Class performance summary
- Subject-wise performance analysis
- Top 5 performer identification
- Attendance analysis
- Student ID-based search
- Interactive terminal menu
- Separate functions for calculations and analysis
- Test file for validating the student metric calculations

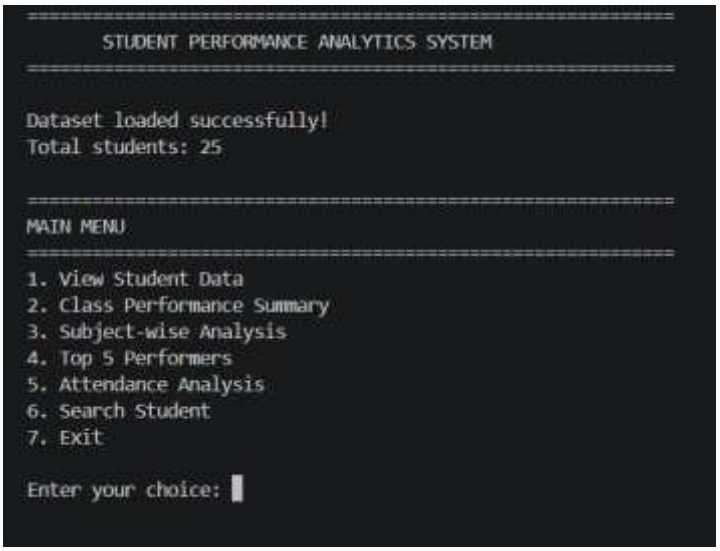
7. Output / Screenshots

The project was tested through the interactive terminal application.

The following screenshots demonstrate the major outputs of the system:

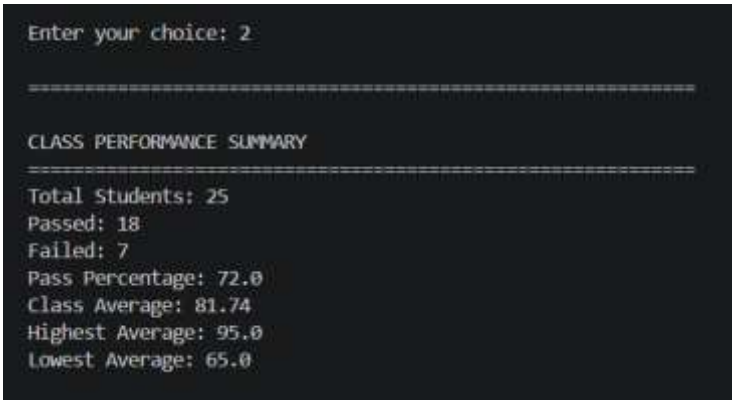
Screenshot 1 – Main Menu

Shows the application starting successfully and displaying all available analysis options.



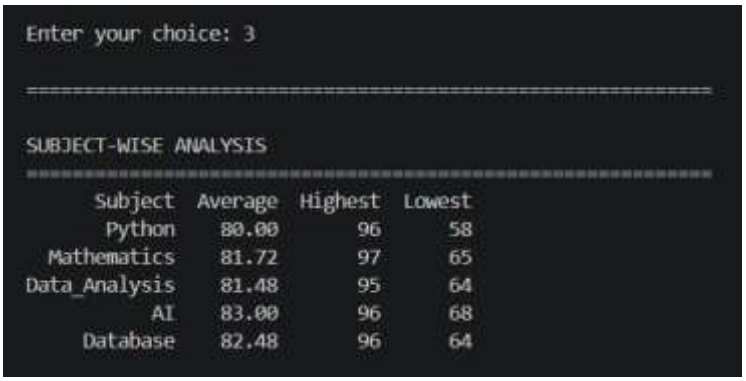
Screenshot 2 – Class Performance Summary

Shows the total number of students, passed students, failed students, pass percentage and class performance statistics.



Screenshot 3 – Subject-wise Analysis

Shows the average, highest and lowest marks for each subject.



Screenshot 4 – Top 5 Performers

Shows the five students with the highest average marks.

```
Enter your choice: 4

============================================================

TOP 5 PERFORMERS
============================================================
Student_ID            Name Department  Average Grade Result
     ST004    Ananya Ghosh        CSE     95.0    A+   Pass
     ST025       Aditi Das        CSE     93.2    A+   Pass
     ST012        Ira Saha         IT     93.2    A+   Pass
     ST008 Riya Chatterjee        ECE     90.6    A+   Pass
     ST016     Soham Mitra        CSE     90.4    A+   Pass
```

Screenshot 5 – Attendance Analysis

Shows attendance-related statistics and students below the required attendance percentage.

```
Enter your choice: 5

============================================================

ATTENDANCE ANALYSIS
============================================================
Average Attendance: 81.68
Highest Attendance: 96
Lowest Attendance: 62
Students Below 75%: 7
```

Screenshot 6 – Student Search

Shows the result of searching for an individual student using Student ID.

```
Enter your choice: 6

Enter Student ID: ST004

============================================================

STUDENT PERFORMANCE
============================================================
Student_ID        Name Department  Attendance  Python  Mathematics  Data_Analysis  AI  Database  Total  Average Grade Result
     ST004 Ananya Ghosh        CSE          96      94           97             95  93        96    475     95.0    A+   Pass
```

8. Final Outcome

The Student Performance Analytics System was successfully implemented and tested using the student dataset.

The final analysis produced the following results:

Metric	Result
Total Students	25
Passed Students	18
Failed Students	7
Pass Percentage	72.0%
Overall Class Average	81.74
Metric	**Result**
Highest Student Average	95.0
Lowest Student Average	65.0

Top 5 Performers

Rank	Student	Department	Average	Grade
1	Ananya Ghosh	CSE	95.0	A+
2	Ira Saha	IT	93.2	A+
3	Aditi Das	CSE	93.2	A+
4	Riya Chatterjee	ECE	90.6	A+
5	Soham Mitra	CSE	90.4	A+

Subject-wise Results

Subject	Average	Highest	Lowest
Python	80.00	96	58
Mathematics	81.72	97	65
Data Analysis	81.48	95	64
AI	83.00	96	68
Database	82.48	96	64

The analysis shows that **AI has the highest subject average at 83.00** among the five subjects.

Overall, the project successfully demonstrates how Python, NumPy and Pandas can be used to transform raw student records into meaningful academic performance insights.

9. Challenges & Learning

Challenges Faced

One of the initial challenges was setting up the required Python libraries for the project.

Another challenge was organizing the program so that calculations could be reused instead of writing the same logic repeatedly.

Testing the different menu options was also important to make sure that the calculations and displayed results were correct.

How the Challenges Were Solved

The required libraries were installed using Python's package manager.

The project was separated into:

- `main.py` for running the application
- `functions.py` for reusable calculations and analysis
- `students.csv` for the dataset
- `test_functions.py` for testing

The application was then tested through its different menu options and the resulting values were checked.

Learning Outcomes

Through this project, I learned and practiced:

- Reading CSV data using Pandas.
- Working with Pandas DataFrames.
- Performing numerical calculations using NumPy.

- Creating reusable Python functions.
- Using loops for repeated processing.
- Using conditional statements for decision-making.
- Filtering and sorting DataFrame records.
- Performing basic data analysis.
- Designing a menu-driven Python application.
- Testing and debugging a Python project.
- Organizing a Python project for GitHub submission.

This project helped strengthen my understanding of how Python programming concepts can be combined with data-analysis libraries to build a practical application.

10. GitHub Repository

The complete project is available on GitHub.

Repository:

https://github.com/Uhan-ara/student-performance-analytics

The repository contains:

- `main.py`
- `functions.py`
- `students.csv`
- `test_functions.py`
- `requirements.txt`
- `README.md`
- `screenshots/`

The repository contains the source code, dataset, documentation resources and project screenshots required for submission.

Conclusion

The Student Performance Analytics System successfully meets the main requirements of the Python project by combining Python fundamentals with NumPy and Pandas.

The project processes 25 student records and provides student-level calculations, class performance statistics, subject-wise analysis, attendance analysis, top-performer identification and student search functionality.

The completed project demonstrates practical application of **Python functions, loops, conditions, NumPy, Pandas and data analysis** in a structured and user-friendly system.